ESP32-BASED ELECTRONIC VOTING MACHINE

PROJECT REPORT

An educational prototype for button-based vote input, indicator feedback, and vote tally display

Report details	Information
Submitted by	_____
Department / Class	_____
Submitted to	_____
Submission date	_____

Prototype status: ESP32 controller boot and scanner execution were observed; final LCD detection and main-sketch flash verification remain unresolved.

Prepared from the development details and serial output shared during project troubleshooting.

Abstract

This project presents an ESP32-based Electronic Voting Machine (EVM) prototype intended to demonstrate candidate selection using push buttons, visual feedback through LEDs, and vote totals on a 16×2 I2C LCD. The ESP32 acts as the central controller and reads digital inputs from candidate and results buttons. Development and testing were performed in stages. The ESP32 was successfully programmed with a scanner sketch and booted into its application. The I2C scanner completed its scan but did not report an LCD address in the observed configuration. The main EVM program compiled, while a later upload attempt reported a flash communication warning and an MD5 verification mismatch. This report separates verified results from remaining integration work and does not claim full-system validation.

1. Introduction

Electronic voting systems can simplify the recording and tallying of votes in controlled demonstrations. This project uses an ESP32 development board to implement a small educational voting interface. Each candidate is represented by a push button. A button press is intended to increment the candidate's counter, activate its indicator LED, and provide a confirmation message on the LCD. A separate results button is intended to display the current tally.

The prototype is educational only. It is not designed for official elections and does not provide voter authentication, ballot secrecy protections, tamper resistance, or audited persistent storage.

2. Objectives

- Use an ESP32 as the controller for a simple electronic voting demonstration.
- Read candidate-selection inputs from push buttons.
- Provide visual feedback using candidate indicator LEDs.
- Use a 16×2 I2C LCD to show status and vote totals.
- Develop and test the system in stages and document observed integration issues.

3. Components Used

Component	Purpose
ESP32 development board (30-pin class)	Main controller for the voting logic.
16×2 LCD with I2C backpack	Intended to display status, confirmation, and results.
Four push buttons	Three candidate inputs and one results input.
Three LEDs and 220Ω–330Ω resistors	Candidate indicator feedback with current limiting.
MB-102 breadboard(s), jumper wires, USB cable	Circuit prototyping, connections, and programming.

4. System Design

Intended functional flow:

Candidate button → ESP32 input → vote counter update → LED feedback → LCD message

Results button → ESP32 → current vote totals → LCD / serial output

Button inputs are configured with internal pull-up resistors (INPUT_PULLUP). In this arrangement, a button is considered pressed when its input is connected to ground. The program uses edge detection and a debounce interval to reduce repeated counts caused by switch contact bounce.

5. Pin Assignment (Program Configuration)

Function	ESP32 GPIO
LCD SDA / SCL	GPIO 21 / GPIO 22
Candidate 1 / 2 / 3 buttons	GPIO 18 / GPIO 19 / GPIO 23
Results button	GPIO 25
Candidate 1 / 2 / 3 LEDs	GPIO 26 / GPIO 27 / GPIO 32

The pin list reflects the software configuration shared during development. The physical wiring should be checked against the actual assembled circuit.

6. Software and Operating Principle

The firmware is written in Arduino C++ using the Wire library for I2C communication and LiquidCrystal_I2C for the LCD. During setup, GPIO modes are configured, the LCD is initialized, and a ready message is requested. During the main loop, the controller monitors button states. A detected press increments the relevant counter, updates the corresponding LED, and issues a confirmation message. The results button requests display of the accumulated totals.

The vote counters are held in RAM variables and are cleared when the board resets or loses power. This is a limitation of the current educational prototype.

7. Development and Test Summary

Test / observation	Observed outcome
Scanner sketch upload	Successful on alternate breadboard setup.
ESP32 boot	Normal POWERON_RESET / SPI_FAST_FLASH_BOOT output observed.
I2C scanner	Executed; no LCD address was printed.
Main EVM sketch compilation	Successful; approximately 296,472 bytes (22%) program storage.
Main sketch flash verification	Not successful; flash communication warning and MD5 mismatch.
Complete voting cycle / LCD	Not confirmed end-to-end.

8. Serial Output Evidence

The following excerpts are formatted as terminal output from the reported development session. They reproduce the relevant messages shared for this report; they are evidence of the observed test stages, not a claim that the complete device passed all tests.

8.1 Successful boot and scanner execution

```
rst:0x1 (POWERON_RESET),boot:0x13 (SPI_FAST_FLASH_BOOT)
configsip: 0, SPIWP:0xee
clk_drv:0x00,q_drv:0x00,d_drv:0x00,cs0_drv:0x00,d0_drv:0x00
mode:DIO, clock div:1
entry 0x4008059c

I2C Scanner Started
Scan complete
```

Interpretation: The ESP32 booted and executed the scanner program. Since no address was printed between scanner start and completion, the LCD was not detected on the configured I2C bus during this scan.

8.2 Main EVM sketch compilation

```
WARNING: library LiquidCrystal I2C claims to run on avr
architecture(s) and may be incompatible with your current
board which runs on esp32 architecture(s).

Sketch uses 296472 bytes (22%) of program storage space.
Global variables use 23532 bytes (7%) of dynamic memory.
```

Interpretation: The code compiled. The library architecture message is a compatibility warning; it is not itself proof of an upload failure.

8.3 Flash write / verification error

```
Warning: Failed to communicate with the flash chip,
read/write operations will fail. Try checking the chip
connections or removing any other hardware connected to IOs.

Writing at 0x00001000 ... 100.0%
Wrote 23488 bytes (15089 compressed) at 0x00001000.
Verifying written data...

A fatal error occurred:
MD5 of file does not match data in flash!
Failed uploading: uploading error: exit status 2
```

Interpretation: The uploader began writing data, but the subsequent flash verification failed. Therefore, this main-sketch upload is recorded as unverified / unsuccessful.

9. Limitations

- The complete hardware system has not been verified in an end-to-end test.
- The I2C scanner did not detect an LCD address in the reported configuration.
- The main sketch upload encountered a flash communication warning and MD5 verification error.
- Vote counts are stored in RAM and are cleared on reset or power loss.
- No voter authentication, ballot privacy, cryptographic protection, or tamper-evident audit trail is implemented.
- This prototype is suitable for educational demonstration only, not official election use.

10. Future Improvements

- Stabilize flash programming with external connections disconnected, then reconnect peripherals individually.
- Verify LCD power, common ground, SDA/SCL routing, and its I2C address.
- Test each button and LED independently before the full voting cycle.
- Consider persistent storage and integrity checks for retained demonstration counts.
- Improve access control and auditability in any more advanced educational model.

11. Conclusion

An ESP32-based EVM prototype was designed and its firmware was developed. The development process verified that the ESP32 could be programmed with a scanner sketch and boot into application code. The main EVM sketch compiled successfully, but the final upload was not verified because the uploader reported a flash communication warning and an MD5 mismatch. The I2C scan also completed without detecting the LCD. The project demonstrates the proposed controller-based voting approach at the software and prototype-design level, while full LCD integration and reliable final hardware operation remain future work.

Appendix A — Pin Map

LCD: SDA GPIO21, SCL GPIO22. Buttons: Candidate 1 GPIO18, Candidate 2 GPIO19, Candidate 3 GPIO23, Results GPIO25. LEDs: Candidate 1 GPIO26, Candidate 2 GPIO27, Candidate 3 GPIO32. With INPUT_PULLUP, each button connects between its GPIO and GND. Each LED requires a series current-limiting resistor.

Status summary: Successful scanner upload and normal boot observed; main EVM sketch compilation successful; LCD address detection and final flash verification not successful / pending.